



COMP3006 Designing Intelligent Agents

Author	Chua Xin Hui
Student ID	20219680
Email	hcyxc1@nottingham.ac.uk
Programme of Study	Computer Science with AI (BSc)

Submission Date: 10th May 2022

Table of Contents

1. Project Description -----	pg 2
2. Background & Related Work -----	pg 2-4
3. Design & Implementation -----	pg 4-5
4. Testing -----	pg 5-6
5. Results -----	pg 6-8
6. Discussion -----	pg 8
7. Conclusion -----	pg 9
8. References -----	pg 9

1. Project Description

This project involves robot vacuum cleaners collecting dirt in a simulated environment, which is taken from the labs. The question to be answered is: “How does the bot’s ability to collect dirt change with different movement behaviours?” To answer this question, a few sub questions must first be answered: “What are the movement behaviours tested?”, “Will increasing the number of bots hinder or improve the dirt collecting performance?” and “How is the dirt distributed?”. The behaviours tested involve a collision avoidance mechanism, and a behaviour in which bots are attracted to dirt in its vicinity. To evaluate the performance, the time taken for bots to collect a certain amount of dirt is measured.

2. Background & Related Works

A robot vacuum cleaner AKA robovac refers to autonomous robots that move around and perform cleaning tasks. Some of the widely known robovacs include iRobot’s Roomba and Neato, Ecovac’s Deebot, Dyson’s Heurist, etc. These bots are typically used in household environments for cleaning functionalities like sweeping, vacuuming, even mopping. To carry out these cleaning tasks with minimal (if any) human supervision, these bots are controlled by intelligent programs.

In order for mobile robots to navigate, they first need to have a way of gathering information about their environment (modelling/mapping). This data can be gathered using various methods such as by using optical sensors, infrared sensors, ultrasound sensors, proximity sensors, lasers, cameras, etc. Once the robot has done so, it will be able to identify its relative position to the obstacles around it (localization) in order to navigate its way around them. This problem is often referred to as ‘simultaneous localization and mapping’ (SLAM). After that, the robot can plan its traversal path accordingly (path planning).

SLAM

Not only are sensors prone to errors and sensitive to noise, dynamic environments also significantly increase the complexity of mapping. Various methods have been proposed to solve the SLAM problem. Some examples include using the extended Kalman filter SLAM (EKF-SLAM), particle filters, and more recently, using cameras and 3D sensors. Of the two commonly used techniques for mapping – point-based matching and feature-based matching, the latter requires significantly less memory, is more efficient, provides better information about the environment, and extracted features (corners and line segments) can be scaled down; thus, it has been extensively researched and employed into SLAM. Mobile robots typically roam on planar surfaces, and they are typically modelled by line segments. Examples of line feature extraction algorithms include range sensors for indoor mapping, split and merge, incremental, Hough transforms, line regression, RANSAC, and expectation-maximization algorithms. [1]

Path Planning

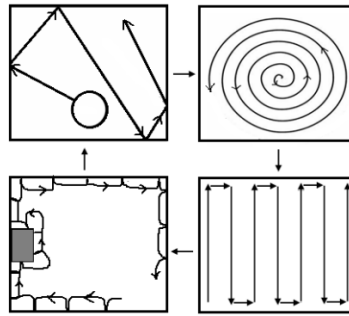
Path planning is arguably the most important task in the navigation process, and has been given the most attention by researchers in the field of autonomous control. It is defined as the determination of a collision-free path in a given environment, which may often be cluttered in a real-world situation [2].

Path planning can either be local or global. Global path planning finds the optimal path for the bot to traverse through and is best performed in static environments that are perfectly known to the robot. Some examples of global path planning algorithms include Dijkstra algorithm, A*, etc. On the other hand, local path planning is performed in dynamic environments, when the robot is moving. Using

data received from the sensors, the bot generates a new path in response to the changes in its environment.

Some examples of state-of-the-art local path planning approaches include D*, Rapidly-exploring Random Trees (RRTs), Potential field method, and hybrid approaches combining Relaxed A* (RA*) with a meta-heuristic algorithm such as GA, Ant Colony or Firefly.

Examples of some basic path planning algorithms include Random Walk, Spiral, 'S' shape pathway, and Wall follow [4], of which the movement pattern is shown below.



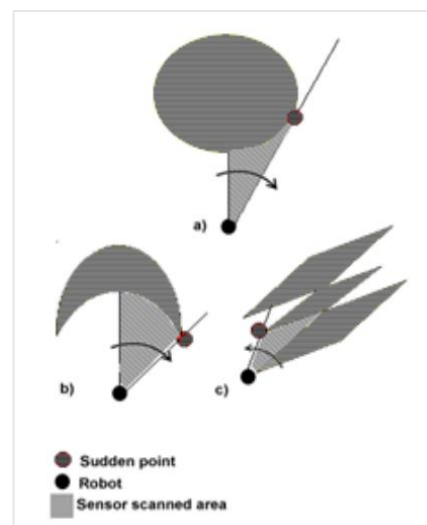
The Collision Avoidance mechanism used in this project works in a similar way as the Random Walk algorithm, but is taken from the Boids algorithm's separation rule, and does not consider the edges of the canvas an obstacle. Random Walk is based on random movement, and does not require precise realization of the route plan [3]. The bot moves forward until an obstacle is sensed and stops if it encounters any barriers. Then based on the sensor readings on the sides of the bot, it chooses a direction to turn towards. A random number is generated to decide how much the bot is to turn, but a condition named corner escape angle is also taken into account to prevent the bot from getting trapped in corners [4][5].

The Move-to-Dirt movement behaviour is inspired by a recently developed path planning algorithm – PointBug algorithm [6].

How PointBug Algorithm Works [6]:

```

While Not Target
  If robot rotation <= 360
    Robot rotates right of left according to position of dmin
    If sudden point
      If 180 degree rotation
        Ignore reading /* to avoid robot return to previous
        point */
      Else
        Get distance from current sudden point to next sudden
        point
        Get angle of robot rotation
        Move to new point according to distance and rotation
        angle
        Record New dmin value
        Reset rotation
      End if
    End if
  Else
    Robot Stop /* No sudden point and exit loop */
  End if
While end
Robot Stop /* Robot successfully reaches target */
  
```



Using data from the sensors - a change in the reading within an interval of time, the bot determines what is considered a sudden point (neighbour). Upon finding a sudden point, the robot rotates just enough until it is directly facing the sudden point in the forward direction. The robot ignores sensor readings at a rotation of 180° or directly behind it, to avoid detection of previous sudden points. If there is no sudden point found within a 360° rotation, the robot will stop.

3. Design & Implementation

Code is written in python language. The environment in which the vacuum cleaning bots act in are taken from the labs. The canvas size is set to 1000x750. Dirt is distributed at random locations on the canvas and a counter is used to keep track of the total dirt collected by the bot(s).

The bot(s) are also placed onto the canvas at random positions and facing a random direction, and are set to continue moving forward in that direction until they are in the vicinity of an obstacle; at which point their behaviour will vary depending on which movement mechanism they were being tested with.

i. Collision Avoidance/Anti-swarm Mechanism

The first movement behaviour tested, is modified from the first rule of Boids' Algorithm for swarm behaviour – the Separation Rule taken from one of the labs. Each bot identifies other bots as its neighbours/nearby bots if they are within a specified distance from each other. A threshold distance is also set to define the region in which bots are considered to be very near, and will label those bots as the nearest bots. If too close to one another, the bots will move away from each other.

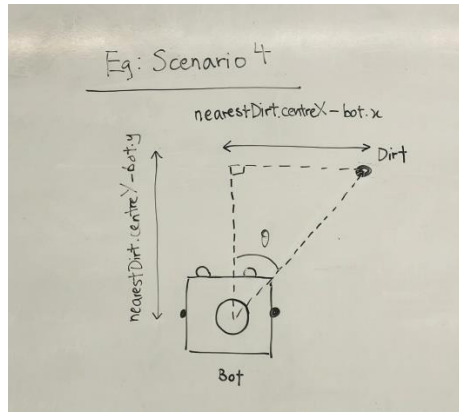
ii. Move Towards Dirt Mechanism

For this movement behaviour, the bot(s) identify a neighbourhood of nearby dirt rather than other bots, and are programmed to orient towards the nearest dirt. As shown in the code and illustrated below, the difference of the x and y distances between each bot and the centre of its nearest dirt is calculated in order to calculate the arctangent angle; the angle in which the bot needs to rotate in order to orient itself towards the dirt.

```

199 # Orient towards nearest dirt
200 if nearestDirt is not None and bots.distanceTo(nearestDirt)<50 :
201
202     #Scenario1. dirt is directly above the bot, on the same x-axis
203     if ( (nearestDirt.centrey-bots.y > 0.0) & (nearestDirt.centrex-bots.x == 0.0) ):
204         bots.theta += math.pi
205
206     #Scenario2. dirt is directly on the right of the bot, on the same y axis
207     if ( (nearestDirt.centrex-bots.x > 0.0) & (nearestDirt.centrey-bots.y == 0.0) ):
208         bots.theta -= (math.pi/2.0)
209
210     #Scenario3. dirt is directly on the left of the bot, on the same y axis
211     if ( (bots.x-nearestDirt.centrex > 0.0) & (nearestDirt.centrey-bots.y == 0.0) ):
212         bots.theta += (math.pi/2.0)
213
214     #Scenario4. dirt is facing top right of the bot on different axes
215     if ( (nearestDirt.centrex-bots.x > 0.0) & (nearestDirt.centrey-bots.y > 0.0) ):
216         bots.theta -= math.atan((nearestDirt.centrex-bots.x)/(nearestDirt.centrey-bots.y))
217
218     #Scenario5. dirt is facing bottom right of the bot on different axes
219     if ( (nearestDirt.centrex-bots.x > 0.0) & (bots.y-nearestDirt.centrey > 0.0) ):
220         bots.theta -= bots.theta + math.atan((bots.y-nearestDirt.centrey)/(nearestDirt.centrex-bots.x))
221
222     #Scenario6. dirt is facing top left of the bot on different axes
223     if ( (bots.x-nearestDirt.centrex > 0.0) & (nearestDirt.centrey-bots.y > 0.0) ):
224         bots.theta += math.atan((bots.x-nearestDirt.centrex)/(nearestDirt.centrey-bots.y))
225
226     #Scenario7. dirt is facing bottom left of the bot on different axes
227     if ( (bots.x-nearestDirt.centrex > 0.0) & (bots.y-nearestDirt.centrey > 0.0) ):
228         bots.theta += (math.pi-bots.theta) - math.atan((bots.y-nearestDirt.centrey)/(bots.x-nearestDirt.centrex))
229
230     #Scenario8. dirt is directly below the bot, on the same x-axis
231     if ( (bots.y-nearestDirt.centrey > 0.0) & (nearestDirt.centrex-bots.x == 0.0) ):
232         bots.theta += math.pi

```



The bots in this mechanism have no collision avoidance mechanism and will have no awareness of each other's position, but will turn towards the nearest dirt in their path (if any).

4. Testing

To answer the questions and evaluate the performances, the program is executed 75 times for each mechanism – 5 times for each number of bots, ranging from 1 to 15 bots. To standardise the testing, the number of dirt is set to 200 for all the runs, and the speed of the bots are fixed. Once the bot(s) have cleaned up all the dirt, meaning the counter has reached 200, the program ends.

Due to the nature of the Anti-swarm mechanism, when testing with 1 bot, the bot has a possibility of never clearing all the dirt. The bot will never rotate since there are no other bots for it to come in contact with. If the dirt is not in the path at which the angle of the bot is moving towards to begin with, the dirt will never be cleared. When this occurs, the program is simply restarted.

To evaluate the performance of the bots, the time taken for the bots to collect all the dirt is measured. For each run, the elapsed time (in seconds) is calculated by subtracting the time at which the program starts (st) from the time at which the program ends (et-st). The average time taken (of the 5 runs) for each configuration is then calculated.

```

242     if count.dirtCollected == 200:
243         et = time.time()
244         elapsed_time = et-st
245         print('Time taken to collect all dirt:', elapsed_time, 'seconds')
246         sys.exit()
247
248
249 # run in an endless loop
250 def main():
251     window = tk.Tk()
252     canvas = initialise(window)
253     registryActives, registryPassives, count = register(canvas)
254     moveIt(canvas, registryActives, registryPassives, count)
255     window.mainloop()
256
257 st = time.time()
258 main()

```

The results collected are stored in an excel file, and R is used to plot the graphs shown in the Results section below.

R Script

```
library(tidyverse)
library(gcookbook)

#LOAD DATA
results <- read.csv("C:/Users/amand/Desktop/DIA_results2.csv")
view(results)

#Modify data
newresults <- results %>%
  mutate(avrgtime = (Round1+Round2+Round3+Round4+Round5)/5)
view(newresults)

#bar plot
plot1 <- ggplot(data= newresults, aes(x=noOfBots, y=avrgtime, fill=Mechanism)) +
  labs(title="Performance Comparison In Relation To Varying Number
of Bots and Different Movement Mechanism",
x="Number of Bots",
y="Average Time Taken To Clean All Dirt (s)") +
  geom_bar(stat="identity", position = "dodge", width = 0.7)
plot1

#line plot
plot2 <- ggplot(data=newresults, aes(x=NoOfBots, y=avrgtime, group=Mechanism)) +
  geom_line(aes(linetype=Mechanism),size=1.0)+
  geom_point(aes(shape=Mechanism))
plot2

#boxplot
plot3 <- ggplot(data=results, aes(x=Mechanism, y=TimeTaken, fill=Mechanism)) +
  geom_boxplot(notch="True", width=0.5, outlier.colour = "red") +
  theme_minimal()+
  ylim(8,100)+
  labs(title="Performance Comparison of Movement Mechanism",
x="Mechanism",
y="Time Taken To Clean All Dirt (s)") +
  theme(plot.title=element_text(size = 16, face = "bold", hjust = 0.5),
axis.title = element_text(size = 14), axis.text= element_text(size = 12),
legend.title = element_text(size=14), legend.text = element_text(size=12))
plot3
```

5. Results

noOfBots	Round1	Round2	Round3	Round4	Round5	Mechanism	avrgtime
1	213.105486	210.41142	405.12512	284.236567	69.33424	Anti-swarm	236.44257
2	88.314546	123.84997	86.43057	99.417089	87.76309	Anti-swarm	97.15505
3	45.190529	62.79155	68.68367	73.230985	67.35941	Anti-swarm	63.45123
4	42.326422	66.54743	47.46858	45.658866	44.74807	Anti-swarm	49.34987
5	88.008296	51.74797	36.13857	35.750446	87.65330	Anti-swarm	59.85972
6	35.405756	38.37524	52.48262	26.101952	60.53004	Anti-swarm	42.57912
7	33.963999	44.60065	30.64303	41.917676	23.41777	Anti-swarm	34.90863
8	25.913744	27.29326	22.70506	33.402259	26.97698	Anti-swarm	27.25826
9	34.168218	18.07938	31.09158	18.576171	27.15835	Anti-swarm	25.81474
10	32.670010	34.39427	20.99150	37.613707	21.51659	Anti-swarm	29.43722
11	34.399873	26.70942	20.49152	54.324713	30.67316	Anti-swarm	33.31974
12	17.797892	24.54766	24.05588	27.400501	18.90244	Anti-swarm	22.54087
13	27.858814	27.30201	28.32452	21.305171	42.18812	Anti-swarm	29.39573
14	18.261796	16.87445	17.86380	14.710695	15.34146	Anti-swarm	16.61044
15	21.578614	19.05909	20.17547	11.918765	20.97195	Anti-swarm	18.74078
1	159.022709	60.42335	85.32711	112.310272	69.33424	Move to Dirt	97.28353
2	109.043138	82.40110	95.31510	72.687692	96.46870	Move to Dirt	91.18314
3	53.633946	69.28549	64.95125	57.637429	51.04777	Move to Dirt	59.31118
4	50.812996	52.13668	41.39558	60.481345	60.96428	Move to Dirt	53.15818
5	31.518562	45.40121	40.13429	37.884385	44.10072	Move to Dirt	39.80783
6	34.031579	43.70735	45.77493	47.952701	26.56862	Move to Dirt	39.60703
7	33.257698	25.48318	37.64487	22.084701	24.66378	Move to Dirt	28.62685
8	27.973566	22.19191	19.40756	20.179663	24.61276	Move to Dirt	22.87309
9	18.064367	30.32521	15.08967	18.524718	26.53738	Move to Dirt	21.70827
10	16.347195	22.50050	17.76042	15.679316	20.05013	Move to Dirt	18.46751
11	16.851381	15.53015	18.22593	16.306565	13.19874	Move to Dirt	16.02255
12	14.627788	13.42111	16.51720	8.299314	10.25812	Move to Dirt	12.62471
13	11.272155	17.47400	11.92901	15.524785	12.06584	Move to Dirt	13.65316
14	11.980833	23.33785	20.34473	17.699003	14.98612	Move to Dirt	17.66971
15	9.929366	13.43336	13.67016	12.102312	10.75135	Move to Dirt	11.97731

Figure 1: Barchart

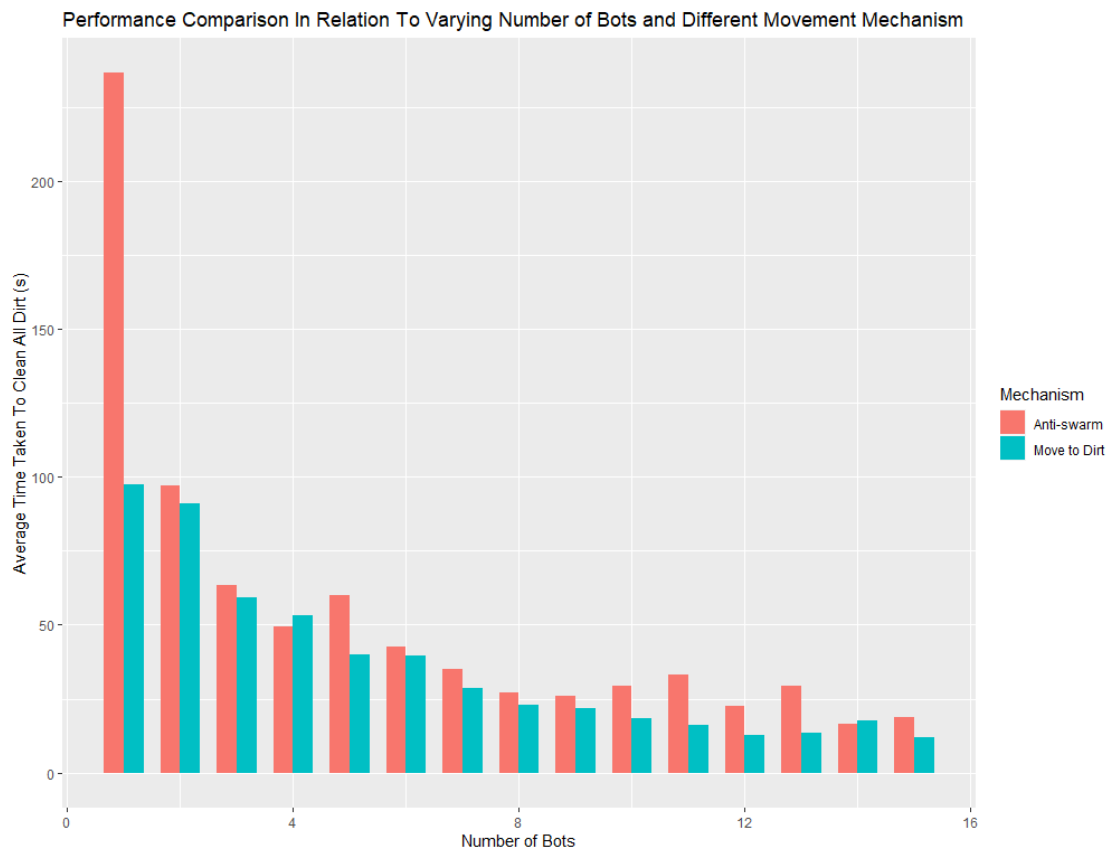


Figure 2: Line Graph

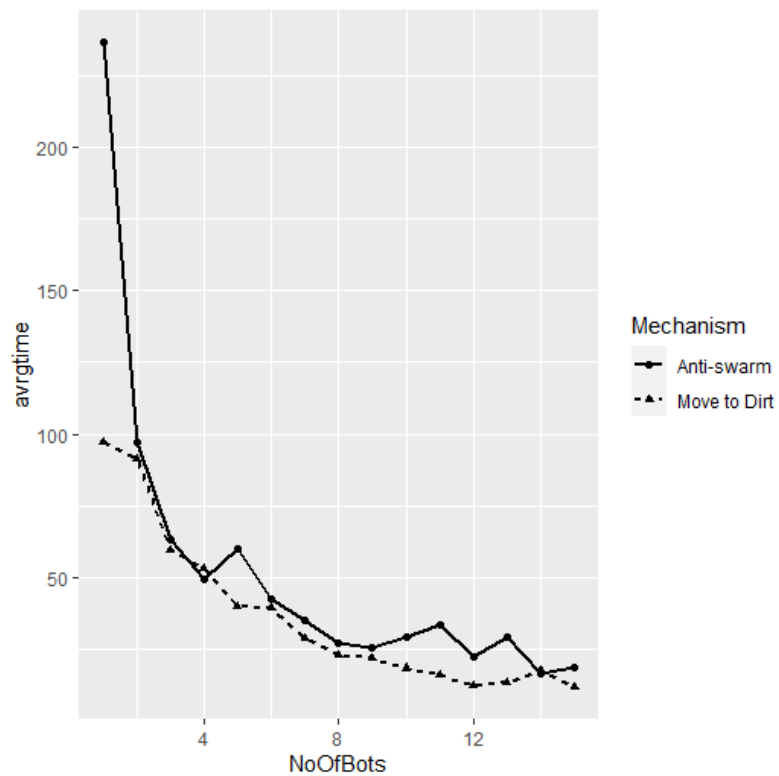
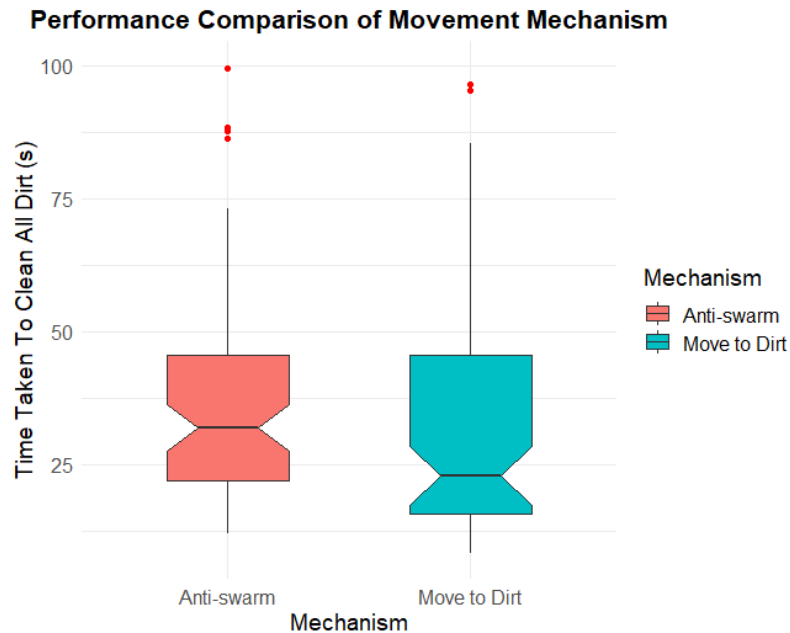


Figure 3: Notched Boxplot



6. Discussion

“How does the bot’s ability to collect dirt change with different movement behaviours?”

Since in the Anti-swarm mechanism, direction of movement of the bots are random, whereas in Move-to-Dirt mechanism, the bots are somewhat guided towards dirt, it is expected that the latter mechanism will perform better than the first, seeing as the latter does not take collision into account.

The results gathered are in line with this expectation. By looking at all 3 figures shown above, we can see that the Move-to-Dirt mechanism performs better overall as compared to the Anti-swarm mechanism, across all numbers of bots. From Figures 1 and 2, only at 4 and 14 bots, the Anti-swarm mechanism performed better, but only by slightly.

“Will increasing the number of bots hinder or improve the dirt collecting performance?”

Since the number of dirt and the size of the canvas remains the same throughout testing, it is fair to expect that as the number of bots increase, the time taken to collect all the dirt would decrease. This is reflected accordingly in the results; as for both mechanisms, the time taken to collect all the dirt is inversely proportional to the number of bots.

As shown in Figures 1 and 2, this difference in performance is especially apparent when testing with only 1 bot, in favour of the Move-to-Dirt mechanism. However, from Figure 2, it becomes apparent that with the anti-swarm mechanism, this may not always be the case; i.e., with 5, 11 and 13 numbers of bots, deviations in the trend are shown.

7. Conclusion

In conclusion, the Move-to-Dirt behaviour generally performs better than the Anti-swarm behaviour; especially if only 1 bot is to be used.

For the most part, the mechanisms were successfully implemented. However, the bots occasionally display unexpected behaviour. For example, for the Anti-swarm mechanism, bots sometimes do not sense other nearby bots until they cross paths at the same position, at which point they will circle around each other for awhile until they eventually separate into different directions. For the Move-to-Dirt mechanisms, bots sometimes rotate too far away from intended direction; i.e., towards the detected dirt, and end up dodging it rather than moving towards it.

With more time, perhaps a movement behaviour where bots move towards dirt whilst also avoiding collision with each other could be implemented. Besides that, the borders of the canvas can be treated as walls in a room, and the bots should not be allowed to go through it and reappear on the other side like how it does now. The environment itself could also be improved on, by using Rviz and Gazebo to generate and view a 3D simulation of a space along with a 3D bot, rather than the current 2D simulation, to better emulate a real-world scenario. In reality, the bot would not have complete environmental knowledge and its relative position within it, so the problem of SLAM will have to be addressed, perhaps using feature extraction followed by line feature extraction algorithms. Furthermore, better local path planning approaches such as Bug Algorithms or PointBug algorithm can be implemented so that the bots can avoid collision with obstacles and react to changes in the environment by generating a new path in real time.

8. References

1. Ravankar, A., Hoshino, Y., Ravankar, A., Jixin, L., Emaru, T., & Kobayashi, Y. (2015). Algorithms and a Framework for Indoor Robot Mapping in a Noisy Environment Using Clustering in Spatial and Hough Domains. *International Journal Of Advanced Robotic Systems*, 12(3), 27. <https://doi.org/10.5772/59992>
2. Mac, T.; Copot, C.; Tran, D.; Keyser, R. Heuristic approaches in robot path planning: A survey. *Robot. Auton. Syst.* 2016, 86. [CrossRef]
3. Stachniss and Cyrill, "Robotic Mapping and Exploration", in Springer Tracts in Advanced Robotics, Vol. 55, 2009, XVIII, 196 p. 89 illus.
4. Y D Zhang, K L Fan, B L Luk, Y H Fung, S K Tso "Mechanical Design of A Cleaning Robot" in 8th IEEE Conference on Mechatronics and Machine Vision in Practice (2001)
5. Ryo Kurazume, Shigeo Hirose, "Development of a Cleaning Robot System with Cooperative Positioning System" in *Autonomous Robots* (2000) Volume 9, Issue: 3, Publisher: Springer, Pages: 237-246
6. Buniyamin, N., Wan Ngah, W., & Mohamad, Z. (2011). A Simple Local Path Planning Algorithm for Autonomous Mobile Robots. *INTERNATIONAL JOURNAL OF SYSTEMS APPLICATIONS, ENGINEERING & DEVELOPMENT*, 5(2), 151-155. Retrieved 10 May 2022, from.